Politécnico de Viseu

Escola Superior de Tecnologia e Gestão de Viseu

Métodos de Procura, OCR e LLM

Unidade Curricular de Inteligência Artificial

Licenciatura em Engenharia Informática

David Simão Borges 27431

Patrícia Isabel Gaspar Oliveira 22525

Susana Daniela Tavares 27467

abril de 2026

Politécnico de Viseu
Escola Superior de Tecnologia e Gestão de Viseu

departamento de informática

Métodos de Procura, OCR e LLM

Unidade Curricular de Inteligência Artifical

Licenciatura em Engenharia Informática

David Simão Borges 27431

Patrícia Isabel Gaspar Oliveira 22525

Susana Daniela Tavares 27467

abril de 2026

Resumo

Este trabalho apresenta o desenvolvimento de um sistema de apoio ao planeamento de rotas entre cidades portuguesas, combinando métodos clássicos de procura em Inteligência Artificial com funcionalidades de OCR e integração com um modelo de linguagem local. Foram implementados os algoritmos Custo Uniforme, Profundidade Limitada, Procura Sôfrega e A*, com registo de caminho, custo total e métricas de execução. O sistema integra ainda autenticação por matrícula através de OCR e um módulo LLM para sugestão de atrações turísticas por cidade. A solução foi disponibilizada numa arquitetura web com frontend e backend, permitindo execução interativa, comparação de algoritmos e exportação de resultados. Os resultados experimentais mostram diferenças relevantes entre algoritmos ao nível de custo, tempo e número de nós expandidos, confirmando o comportamento esperado pela teoria.

Palavras-chave: Inteligência Artificial, Métodos de Procura, A*, OCR, LLM, Planeamento de Rotas.

Índice

1 Introdução 11
1.1 Objetivos 11
1.2 Estrutura do Relatório 11
2 Enquadramento Teórico 12
2.1 Procura Cega 12
2.2 Procura Heurística 12
2.3 Admissibilidade e Consistência 12
2.4 OCR e LLM 12
3 Arquitetura e Modelação do Problema 13
3.1 Arquitetura do Sistema 13
3.2 Modelação em Grafo 15
3.3 Formulação do Problema 16
3.4 Heurística 17
4 Implementação 18
4.1 Algoritmos de Procura 18
4.2 OCR de Matrícula 21
4.3 Integração com LLM Local 23
4.4 Backend e Frontend 23
5 Metodologia Experimental 26
5.1 Desenho Experimental 26
5.2 Cenários de Teste 26
5.3 Métricas Avaliadas 26
5.4 Procedimento de Recolha 27

6 Resultados 28
6.1 Comparação entre Algoritmos 28
6.2 Exemplo de Tabela de Resultados 28
6.3 OCR 29
6.4 LLM 29
6.5 Exportação JSON e CSV 29
7 Discussão 31
8 Conclusões e Trabalho Futuro 32
Referências Bibliográficas 33

Índice de Figuras

Figura 1 - Arquitetura cliente-servidor com separação entre interface, API e módulos de IA. 13

Figura 2 - Fluxo de utilização desde autenticação por matrícula até análise de resultados. (1) 14

Figura 3 - Fluxo de utilização desde autenticação por matrícula até análise de resultados. (2) 15

Figura 4 - Representação do problema em grafo ponderado de cidades portuguesas. 16

Figura 5 - Definição da heurística utilizada pelos métodos informados. 16

Figura 6 - Implementação do algoritmo Custo Uniforme.(1) 18

Figura 7 - Implementação do algoritmo Custo Uniforme. (2) 19

Figura 8 - Implementação do algoritmo de Profundidade Limitada. 19

Figura 9 - Implementação da Procura Sôfrega baseada em h(n). 20

Figura 10 - Implementação do A* com função f(n)=g(n)+h(n). (1) 20

Figura 11 - Implementação do A* com função f(n)=g(n)+h(n). (2) 21

Figura 12 - Integração LLM local 22

Figura 13 - Normalização e validação de matrícula portuguesa no backend 22

Figura 14 - Leitura de matrícula por OCR 22

Figura 15 - Chamada ao LLM e resposta 23

Figura 16 - Interface do website 24

Figura 17 - Interface de configuração da rota 25

Figura 18 - Comparação de algoritmos 25

Figura 19 - Exportação JSON 30

Figura 20 - Exportação CSV 30

Índice de Tabelas

Tabela 1 - Tabela de Resultados - Porto -> Viseu 28

1 Introdução

A procura de caminhos em grafos é um problema clássico da Inteligência Artificial, com aplicações em navegação, logística e sistemas de apoio à decisão. Neste contexto, o presente projeto propõe uma solução prática para encontrar rotas entre cidades portuguesas através de diferentes algoritmos de procura. Para além do núcleo algorítmico, o sistema integra reconhecimento ótico de caracteres (OCR) para validação de matrícula e um modelo de linguagem local (LLM) para enriquecer a experiência do utilizador com informação turística contextual.

A principal motivação foi construir uma solução completa, não apenas do ponto de vista teórico, mas também do ponto de vista de engenharia de software, com interface web, API e validação experimental. O relatório descreve a modelação do problema, a implementação, os testes e a análise comparativa dos resultados.

1.1 Objetivos

Os objetivos deste projeto foram:

- Implementar os algoritmos Custo Uniforme, Profundidade Limitada, Procura Sôfrega e A*.
- Modelar o problema como um grafo de cidades com custos de deslocação.
- Integrar OCR para leitura e validação de matrícula.
- Integrar LLM local para geração de informação turística.
- Desenvolver interface web para execução e comparação dos algoritmos.
- Avaliar experimentalmente desempenho e qualidade das soluções obtidas.

1.2 Estrutura do Relatório

O relatório está organizado em enquadramento teórico, arquitetura e modelação, implementação, metodologia experimental, resultados, discussão, conclusões e trabalho futuro.

2 Enquadramento Teórico

2.1 Procura Cega

A procura cega não utiliza informação sobre o objetivo. No Custo Uniforme, a expansão é feita pelo menor custo acumulado (g(n)), garantindo solução ótima para custos não negativos. Na Profundidade Limitada, a exploração segue uma estratégia em profundidade com limite máximo, sendo sensível ao valor desse limite.

2.2 Procura Heurística

A procura heurística usa uma estimativa (h(n)) do custo até ao objetivo. A Procura Sôfrega expande o nó com menor (h(n)), ignorando o custo acumulado. O A* usa (f(n)=g(n)+h(n)), combinando custo real e estimativa.

2.3 Admissibilidade e Consistência

Uma heurística é admissível se nunca sobrestima o custo ótimo e consistente se satisfaz a desigualdade triangular ao longo das arestas. Estas propriedades são relevantes para garantir a otimalidade do A*.

2.4 OCR e LLM

OCR permite extrair texto de imagens, sendo usado neste trabalho para reconhecer matrículas. O LLM local permite gerar descrições contextuais sem dependência de serviços externos cloud.

3 Arquitetura e Modelação do Problema

3.1 Arquitetura do Sistema

A solução segue arquitetura cliente-servidor:

- Frontend web para interação com o utilizador.
- Backend API para orquestração da lógica.
- Módulos de IA para procura, OCR e LLM.

Esta separação promove modularidade, reutilização de código e facilidade de manutenção.

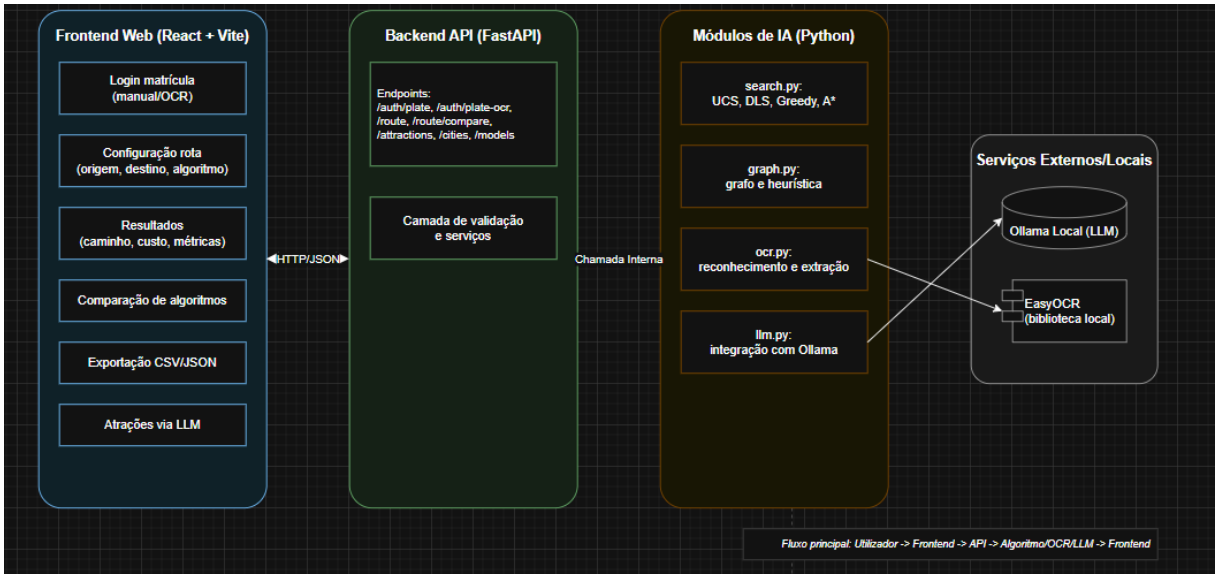

Figura 1 - Arquitetura cliente-servidor com separação entre interface, API e módulos de IA.

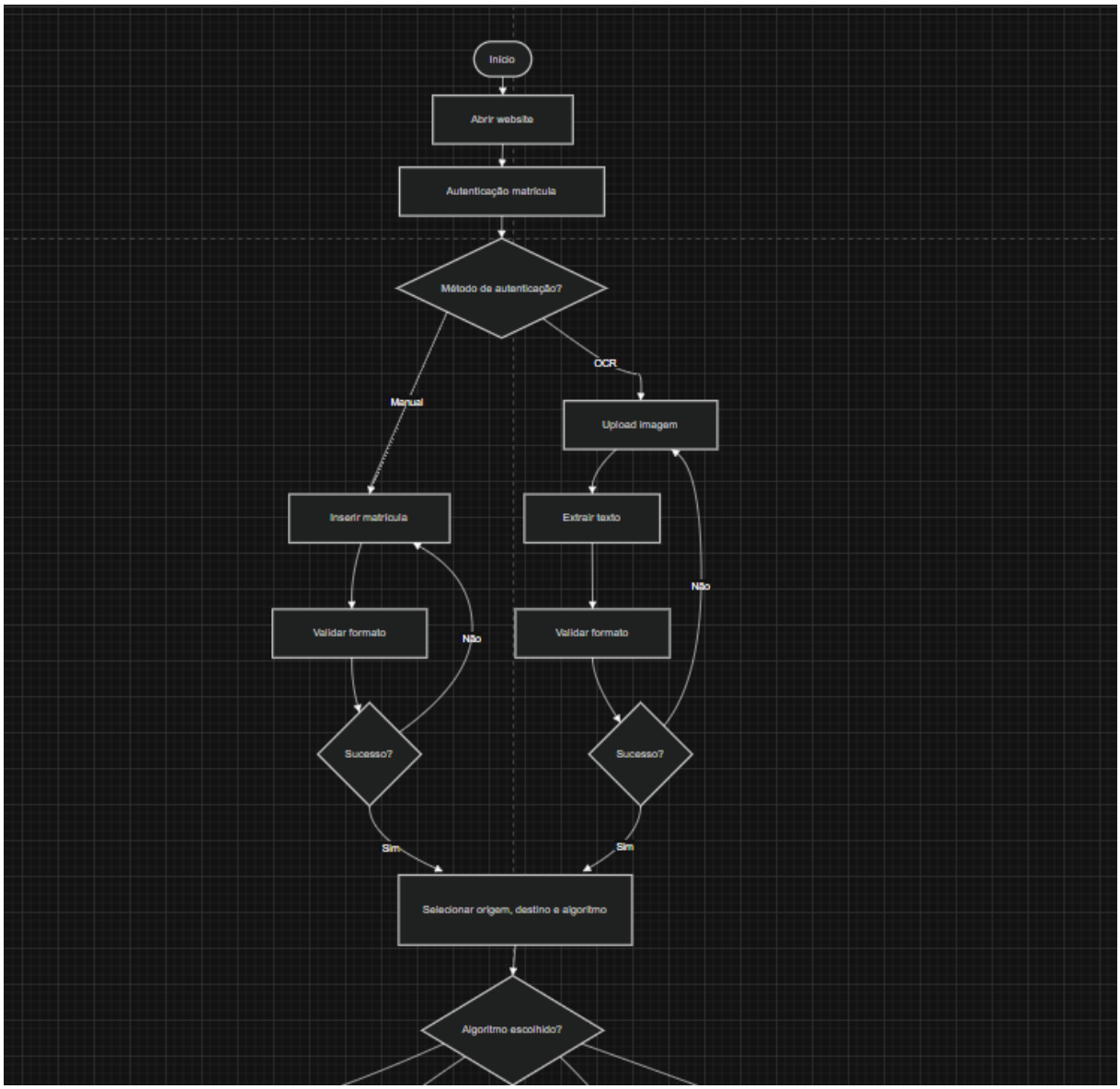

Figura 2 - Fluxo de utilização desde autenticação por matrícula até análise de resultados. (1)

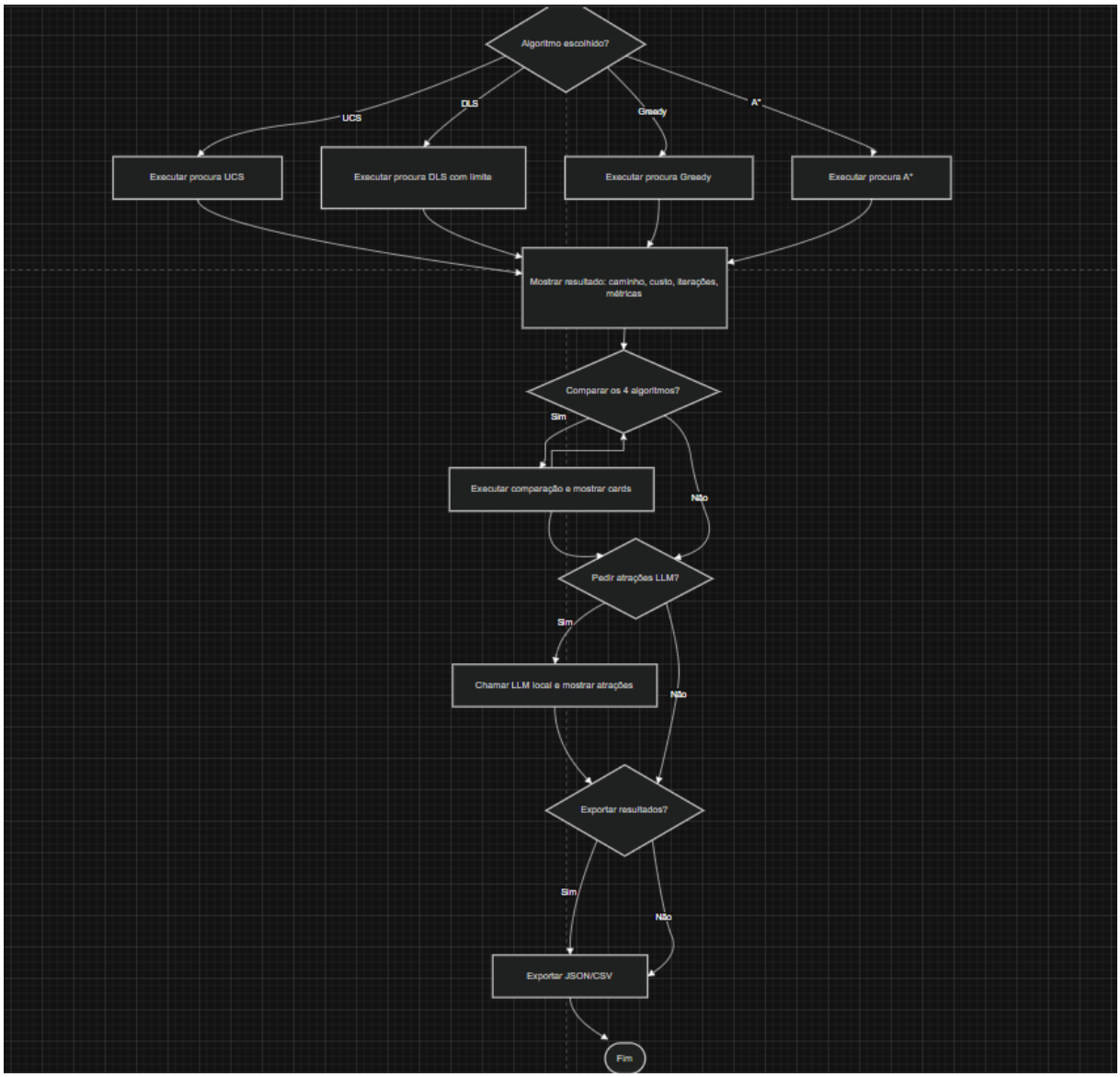

Figura 3 - Fluxo de utilização desde autenticação por matrícula até análise de resultados. (2)

3.2 Modelação em Grafo

O domínio foi modelado como um grafo ponderado e bidirecional:

- Cada cidade corresponde a um nó.
- Cada ligação entre cidades corresponde a uma aresta.
- O peso da aresta representa distância em quilómetros.
- O custo de um caminho é a soma dos pesos das arestas percorridas.

```
GRAPH = {
    'Aveiro':         {'Porto': 68,   'Viseu': 95,  'Coimbra': 68,  'Leiria': 115},
    'Braga':          {'Viana do Castelo': 48, 'Vila Real': 106, 'Porto': 53},
    'Bragança':       {'Vila Real': 137, 'Guarda': 202},
    'Beja':           {'Évora': 78,   'Faro': 152,  'Setúbal': 142},
    'Castelo Branco': {'Coimbra': 159,'Guarda': 106, 'Portalegre': 80, 'Évora': 203},
    'Coimbra':        {'Aveiro': 68,  'Viseu': 96,  'Leiria': 67,   'Castelo Branco': 159},
    'Évora':          {'Lisboa': 150, 'Santarém': 117, 'Portalegre': 131, 'Setúbal': 103, 'Beja': 78, 'Castelo Branco': 203},
    'Faro':           {'Setúbal': 249,'Lisboa': 299, 'Beja': 152},
    'Guarda':         {'Vila Real': 157, 'Viseu': 85, 'Bragança': 202, 'Castelo Branco': 106},
    'Leiria':         {'Lisboa': 129, 'Santarém': 70, 'Aveiro': 115, 'Coimbra': 67},
    'Lisboa':         {'Santarém': 78,'Setúbal': 50, 'Leiria': 129,  'Faro': 299,  'Évora': 150},
    'Portalegre':     {'Castelo Branco': 80, 'Évora': 131},
    'Porto':          {'Aveiro': 68,  'Braga': 53,  'Viana do Castelo': 71, 'Vila Real': 116, 'Viseu': 133},
    'Santarém':       {'Leiria': 70,  'Lisboa': 78, 'Évora': 117},
    'Setúbal':        {'Lisboa': 50,  'Évora': 103, 'Beja': 142,    'Faro': 249},
    'Viana do Castelo': {'Braga': 48, 'Porto': 71},
    'Vila Real':      {'Braga': 106,  'Bragança': 137, 'Guarda': 157, 'Porto': 116, 'Viseu': 110},
    'Viseu':          {'Aveiro': 95,  'Coimbra': 96, 'Guarda': 85,   'Porto': 133, 'Vila Real': 110},
}
```

Figura 4 - Representação do problema em grafo ponderado de cidades portuguesas.

```
HEURISTIC_FARO = {
    'Aveiro':           366,
    'Braga':            454,
    'Bragança':         487,
    'Beja':             99,
    'Castelo Branco':   280,
    'Coimbra':          319,
    'Évora':            157,
    'Faro':             0,
    'Guarda':           352,
    'Leiria':           278,
    'Lisboa':           195,
    'Portalegre':       228,
    'Porto':            418,
    'Santarém':         231,
    'Setúbal':          168,
    'Viana do Castelo': 473,
    'Vila Real':        429,
    'Viseu':            363,
}
```

Figura 5 - Definição da heurística utilizada pelos métodos informados.

3.3 Formulação do Problema

Estado inicial: cidade de origem escolhida pelo utilizador.

Estado objetivo: cidade destino escolhida pelo utilizador.

Ações: transição para qualquer cidade vizinha ligada por aresta.

Função custo: distância total acumulada no percurso.

Critério de sucesso: alcançar o destino com caminho válido; em métodos ótimos, minimizar custo total.

3.4 Heurística

Foi utilizada uma estratégia heurística orientada por distância em linha reta. Para Faro existe uma tabela de referência explícita. Para outros destinos, usa-se aproximação geodésica baseada em coordenadas.

Esta abordagem permite execução informada de A* e Procura Sôfrega em múltiplos cenários, mantendo coerência com os princípios teóricos de estimativa de custo restante.

4 Implementação

4.1 Algoritmos de Procura

Os quatro algoritmos foram implementados com registo de iterações para análise detalhada de execução.

No Custo Uniforme, a fronteira é ordenada por (g(n)), expandindo sempre o custo acumulado mínimo.

Na Profundidade Limitada, a expansão recursiva respeita o limite definido, interrompendo ramos que excedem a profundidade máxima.

Na Procura Sôfrega, a prioridade é definida por (h(n)), privilegiando aproximação heurística imediata.

No A*, a fronteira é ordenada por (f(n)=g(n)+h(n)), equilibrando exploração e direcionamento ao objetivo.

Cada execução devolve: caminho final, custo total, histórico de iterações e métricas de desempenho.

```python
def uniform_cost_search(graph: dict, start: str, goal: str):
    """
    Procura de Custo Uniforme (UCS).
    Expande sempre o nó com menor custo acumulado g(n).

    Parâmetros:
        graph : dicionário de adjacências { cidade: { vizinho: custo } }
        start : cidade de origem
        goal  : cidade de destino

    Retorna:
        path      : lista de cidades do caminho encontrado (ou None)
        cost      : custo total do caminho
        iterations: lista de dicionários com o estado em cada iteração
    """
    # Cada entrada do heap é o tuplo (g, id_unico, caminho).
    # g        = custo acumulado desde a origem até ao nó atual
    # id_unico = contador de desempate (ver nota sobre _counter acima)
    # caminho  = lista de cidades percorridas até ao nó atual
    frontier = [(0, next(_counter), [start])]

    # Conjunto de nós já expandidos. Quando um nó é retirado do heap e já
    # está em explored, é descartado (pode ter entrado várias vezes com
    # custos diferentes antes de ser pela primeira vez expandido).
    explored = set()
    best_cost = {start: 0}

    # Registo de cada nó expandido, guardando o estado nesse momento.
    # Será apresentado ao utilizador para mostrar o funcionamento do algoritmo.
    iterations = []
```

Figura 6 - Implementação do algoritmo Custo Uniforme.(1)

```python
while frontier:
    # Retirar o nó com menor custo acumulado da fila de prioridade
    cost, _, path = heapq.heappop(frontier)
    node = path[-1]  # cidade atual = último elemento do caminho

    # Se este nó já foi expandido com um custo menor, ignorar esta entrada
    if node in explored:
        continue

    # Registar esta expansão no histórico de iterações
    iterations.append({
        'node': node,
        'g':    cost,   # g(n): custo real acumulado desde a origem
        'path': list(path),
    })

    # Verificar se chegámos ao destino
    if node == goal:
        return path, cost, iterations

    # Marcar como expandido para não revisitar
    explored.add(node)

    # Expandir todos os vizinhos ainda não visitados
    for neighbor, edge_cost in sorted(graph.get(node, {}).items()):
        if neighbor in explored:
            continue
        new_cost = cost + edge_cost
        # Só manter na fronteira quando for uma melhoria para o vizinho.
        if new_cost < best_cost.get(neighbor, float('inf')):
            best_cost[neighbor] = new_cost
            heapq.heappush(frontier, (
                new_cost,
                next(_counter),
                path + [neighbor],
            ))

# Fila vazia: não existe caminho entre start e goal
return None, float('inf'), iterations
```

Figura 7 - Implementação do algoritmo Custo Uniforme. (2)

```python
def depth_limited_search(graph: dict, start: str, goal: str, limit: int):
    """
    Procura em Profundidade Limitada (DLS).
    DFS com limite máximo de profundidade.

    Parâmetros:
        graph : dicionário de adjacências
        start : cidade de origem
        goal  : cidade de destino
        limit : profundidade máxima permitida (número de arestas a percorrer)

    Retorna:
        path      : lista de cidades do caminho encontrado (ou None)
        cost      : custo total do caminho
        iterations: lista de dicionários com o estado em cada iteração
    """
    iterations = []

    def _dls(node, path, cost, depth):
        """
        Função recursiva interna.
        node  : cidade atual
        path  : caminho percorrido até agora (incluindo node)
        cost  : custo acumulado até node
        depth : "combustível" restante — quantas arestas ainda podemos percorrer
        """
        iterations.append({
            'node':  node,
            'depth': limit - depth,  # profundidade atual (0 = origem)
            'g':     cost,
            'path':  list(path),
        })

        # Caso base 1: chegámos ao destino
        if node == goal:
            return path, cost

        # Caso base 2: atingimos o limite de profundidade, parar este ramo
        if depth == 0:
            return None, None

        # Expandir vizinhos em ordem alfabética (para resultados determinísticos)
        for neighbor, edge_cost in sorted(graph.get(node, {}).items()):
            if neighbor not in path:  # evitar ciclos: não voltar a cidades já no caminho atual
                result_path, result_cost = _dls(
                    neighbor,
                    path + [neighbor],   # novo caminho com o vizinho incluído
                    cost + edge_cost,
                    depth - 1,           # consumir uma unidade de profundidade
                )
                # Se encontrou solução neste ramo, propagar para cima
                if result_path is not None:
                    return result_path, result_cost

        # Nenhum vizinho levou ao objetivo — falha neste ramo
        return None, None

    # Iniciar DLS a partir da origem com o limite definido
    result_path, result_cost = _dls(start, [start], 0, limit)
    return result_path, (result_cost if result_cost is not None else float('inf')), iterations
```

Figura 8 - Implementação do algoritmo de Profundidade Limitada.

```python
def greedy_search(graph: dict, start: str, goal: str, heuristic):
    """
    Procura Sôfrega (Greedy Best-First Search).
    Expande o nó com menor valor heurístico h(n).

    Parâmetros:
        graph     : dicionário de adjacências
        start     : cidade de origem
        goal      : cidade de destino
        heuristic : função  h(cidade) -> int  — estimativa do custo até ao destino

    Retorna:
        path      : lista de cidades do caminho encontrado (ou None)
        cost      : custo real total do caminho
        iterations: lista de dicionários com o estado em cada iteração
    """
    # Heap ordenado por h(n). Guardamos também o custo real g para exibição,
    # mas ele NÃO influencia a ordem de expansão neste algoritmo.
    # Tuplo: (h, id_unico, caminho, g_acumulado)
    frontier = [(heuristic(start), next(_counter), [start], 0)]
    explored = set()
    iterations = []

    while frontier:
        # Retirar o nó com menor heurística
        h_val, _, path, cost = heapq.heappop(frontier)
        node = path[-1]

        if node in explored:
            continue

        iterations.append({
            'node': node,
            'h':    h_val,  # h(n): estimativa do custo restante até ao destino
            'g':    cost,   # g(n): custo real percorrido (apenas para exibição)
            'path': list(path),
        })

        if node == goal:
            return path, cost, iterations

        explored.add(node)

        for neighbor, edge_cost in sorted(graph.get(node, {}).items()):
            if neighbor not in explored:
                heapq.heappush(frontier, (
                    heuristic(neighbor),      # prioridade = apenas h(vizinho)
                    next(_counter),
                    path + [neighbor],
                    cost + edge_cost,
                ))

    return None, float('inf'), iterations
```

Figura 9 - Implementação da Procura Sôfrega baseada em h(n).

```python
def a_star_search(graph: dict, start: str, goal: str, heuristic):
    """
    Procura A*.
    Expande o nó com menor f(n) = g(n) + h(n).

    Parâmetros:
        graph     : dicionário de adjacências
        start     : cidade de origem
        goal      : cidade de destino
        heuristic : função  h(cidade) -> int  — estimativa admissível do custo restante

    Retorna:
        path      : lista de cidades do caminho encontrado (ou None)
        cost      : custo real total do caminho (g do nó objetivo)
        iterations: lista de dicionários com o estado em cada iteração
    """
    h0 = heuristic(start)  # heurística do nó inicial
    # Heap ordenado por f(n) = g(n) + h(n).
    # Tuplo: (f, id_unico, g_acumulado, caminho)
    frontier = [(h0, next(_counter), 0, [start])]
    explored = set()
    best_g = {start: 0}
    iterations = []

    while frontier:
        # Retirar o nó com menor f = g + h
        f_val, _, g_cost, path = heapq.heappop(frontier)
        node = path[-1]

        if node in explored:
            continue

        # Recuperar h(n) a partir de f e g para exibição na tabela de iterações
        h_val = f_val - g_cost  # h = f - g
```

Figura 10 - Implementação do A com função f(n)=g(n)+h(n). (1)*

```python
        iterations.append({
            'node': node,
            'f':    f_val,   # f(n) = g(n) + h(n): custo total estimado
            'g':    g_cost,  # g(n): custo real desde a origem
            'h':    h_val,   # h(n): estimativa do custo restante
            'path': list(path),
        })

        if node == goal:
            return path, g_cost, iterations

        explored.add(node)

        for neighbor, edge_cost in sorted(graph.get(node, {}).items()):
            if neighbor in explored:
                continue

            new_g = g_cost + edge_cost
            if new_g >= best_g.get(neighbor, float('inf')):
                continue

            best_g[neighbor] = new_g
            new_h = heuristic(neighbor)
            # f do vizinho: custo real até ele + estimativa dali ao destino
            heapq.heappush(frontier, (
                new_g + new_h,
                next(_counter),
                new_g,
                path + [neighbor],
            ))

    # Heap vazio: não existe caminho entre start e goal
    return None, float('inf'), iterations
```

Figura 11 - Implementação do A com função f(n)=g(n)+h(n). (2)*

4.2 OCR de Matrícula

A componente OCR segue o fluxo:

- Receção da imagem enviada pelo utilizador.
- Extração de texto com OCR.
- Filtragem por padrões de matrícula portuguesa.
- Normalização do formato final.
- Rejeição de entradas inválidas com mensagem explícita.
- Esta validação melhora robustez do processo de autenticação e reduz erros de entrada manual.

```python
# Endpoint da API do Ollama para geração de texto.
# Por defeito o Ollama fica na porta 11434 do localhost.
OLLAMA_URL    = "http://localhost:11434/api/generate"

# Modelo a usar por defeito. Pode ser alterado na chamada de cada função.
# llama3.2 é um modelo leve (3B parâmetros) e rápido, adequado a hardware
# doméstico. Alternativas: mistral, phi3, tinyllama, etc.
DEFAULT_MODEL = "llama3.2"


def _post(payload: dict, timeout: int = 120) -> dict:
    """
    Função auxiliar privada: envia um pedido POST ao servidor Ollama.

    Parâmetros:
        payload : dicionário Python que será serializado para JSON e enviado
        timeout : segundos máximos de espera pela resposta (modelos grandes
                  podem demorar mais tempo a gerar texto)

    Retorna:
        Dicionário Python com a resposta JSON do Ollama.

    Lança:
        urllib.error.URLError se o servidor não estiver acessível.
    """
    data = json.dumps(payload).encode("utf-8")
    req  = urllib.request.Request(
        OLLAMA_URL,
        data=data,
        headers={"Content-Type": "application/json"},  # obrigatório para a API REST
        method="POST",
    )
    with urllib.request.urlopen(req, timeout=timeout) as resp:
        return json.loads(resp.read().decode("utf-8"))
```

Figura 12 - Integração LLM local

```python
def normalize_plate(plate: str) -> str:
    clean = re.sub(r"[-\s]", "", plate).upper()
    if len(clean) != 6:
        raise ValueError("Matrícula inválida. Usa formato tipo AA-00-BB")

    if not any(pattern.match(clean) for pattern in PLATE_PATTERNS):
        raise ValueError("Matrícula inválida. Usa formato tipo AA-00-BB")

    normalized = f"{clean[0:2]}-{clean[2:4]}-{clean[4:6]}"
    return normalized
```

Figura 13 - Normalização e validação de matrícula portuguesa no backend

Figura 14 - Leitura de matrícula por OCR

4.3 Integração com LLM Local

A integração LLM é realizada por chamada local ao modelo, com prompt orientado para descrição concisa de atrações por cidade.

A resposta do modelo é apresentada diretamente na interface, associada ao percurso escolhido.

Foram incluídos mecanismos de fallback para indisponibilidade do modelo ou falhas de comunicação, garantindo comportamento previsível da aplicação.

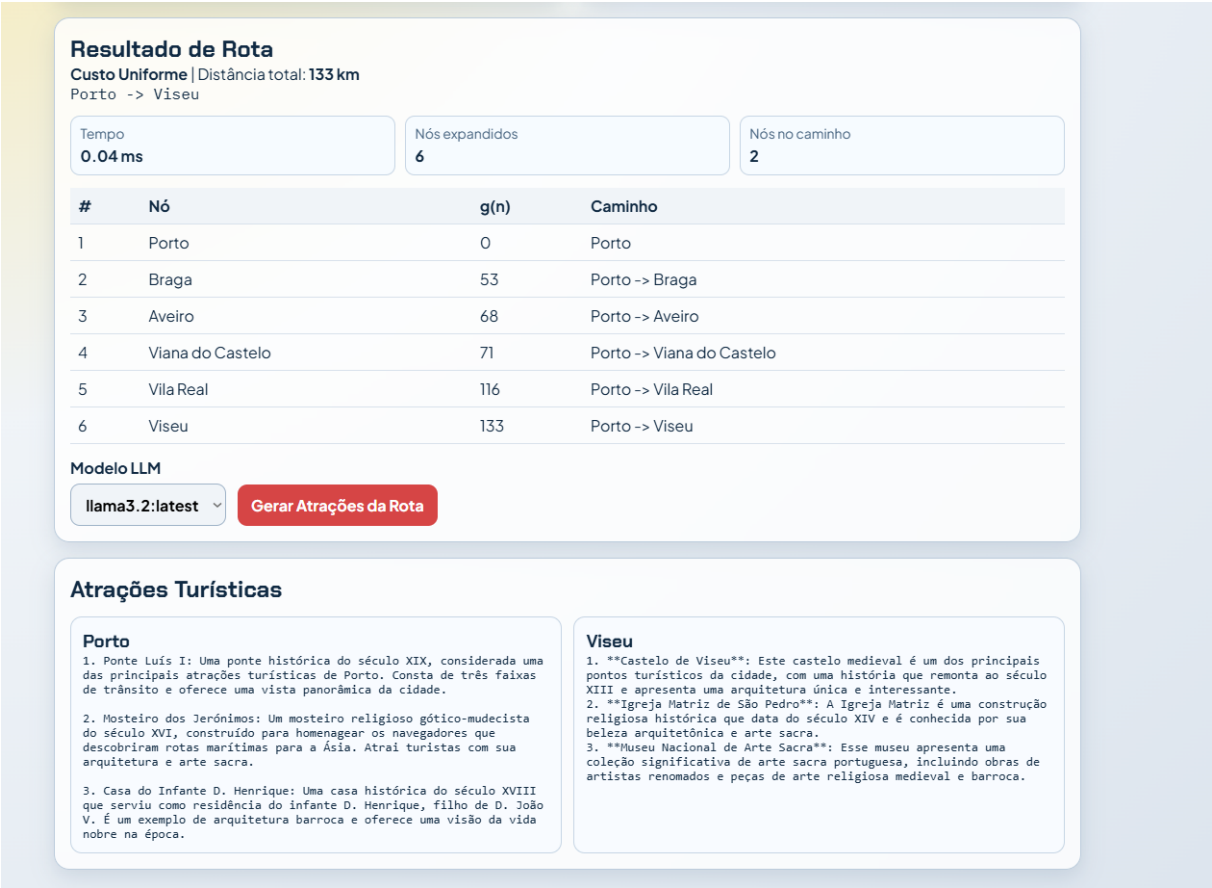

Figura 15 - Chamada ao LLM e resposta

4.4 Backend e Frontend

No backend, a API expõe endpoints para:

- Listagem de cidades.
- Validação manual de matrícula.

- OCR de matrícula via upload.
- Execução de rota por algoritmo.
- Comparação dos quatro algoritmos.
- Pedido de atrações ao LLM.

No frontend, o utilizador consegue:

- Autenticar-se por matrícula manual ou OCR.
- Definir origem, destino e algoritmo.
- Executar rota e analisar tabela de iterações.
- Comparar algoritmos em paralelo.
- Visualizar métricas e exportar resultados.
- Consultar atrações turísticas do caminho calculado.

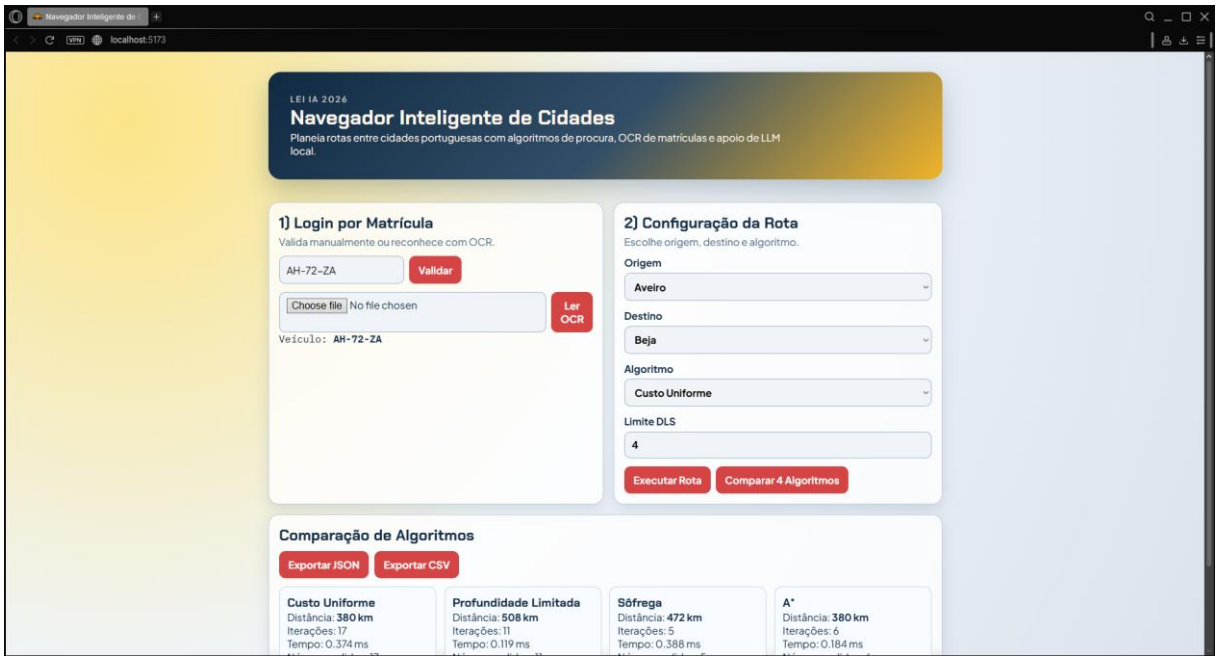

Figura 16 - Interface do website

Figura 17 - Interface de configuração da rota

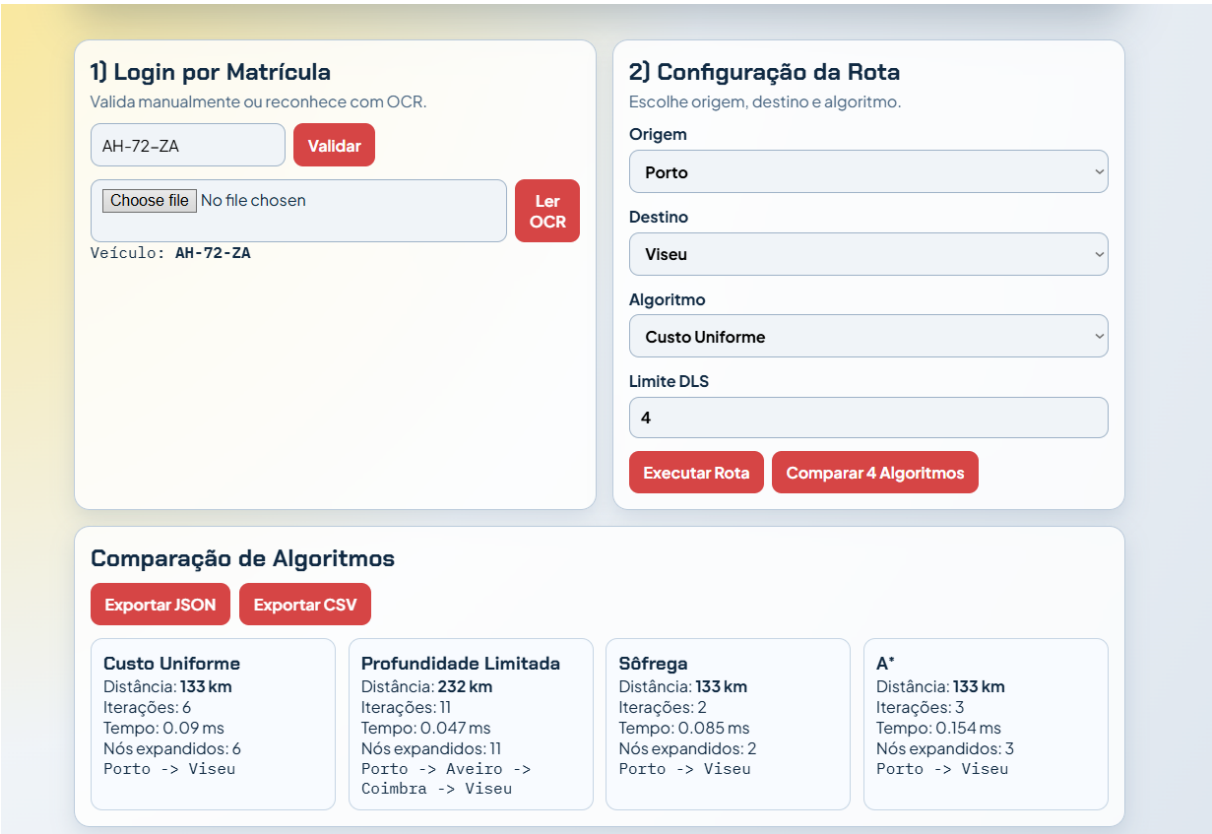

Figura 18 - Comparação de algoritmos

5 Metodologia Experimental

5.1 Desenho Experimental

Para avaliar desempenho e qualidade das soluções, foi definido um conjunto de cenários com diferentes distâncias e topologias de percurso. Cada cenário foi executado com os quatro algoritmos, mantendo condições de entrada comparáveis.

5.2 Cenários de Teste

Os cenários incluíram:

- Percursos curtos entre cidades próximas.
- Percursos médios com múltiplas alternativas.
- Percursos longos entre regiões distantes.
- Casos de sensibilidade para Profundidade Limitada com limites distintos.

5.3 Métricas Avaliadas

As métricas principais foram:

- Custo final do caminho (km).
- Tempo de execução (ms).
- Número de iterações.
- Número de nós expandidos.
- Número de nós no caminho final.
- Taxa de sucesso de OCR por tipo de imagem.
- Coerência qualitativa das respostas LLM.

5.4 Procedimento de Recolha

Para cada execução, os dados foram recolhidos automaticamente pela aplicação e organizados em tabela comparativa. A análise considerou resultados absolutos por cenário e médias agregadas por algoritmo.

6 Resultados

6.1 Comparação entre Algoritmos

Os resultados evidenciaram padrões consistentes com a teoria:

- Custo Uniforme e A* apresentaram, em geral, custos finais equivalentes nos cenários testados.
- A* tendeu a reduzir exploração relativamente a Custo Uniforme em cenários com heurística informativa.
- Procura Sôfrega mostrou menor esforço de exploração em alguns casos, mas com maior variabilidade na qualidade do caminho.
- Profundidade Limitada foi a abordagem mais sensível ao parâmetro de limite.

6.2 Exemplo de Tabela de Resultados

Tabela 1 - Tabela de Resultados - Porto -> Viseu

Cenário	Algoritmo	Custo (km)	Iterações	Nós expandidos	Tempo (ms)
Porto -> Viseu	UCS	133	6	6	0.067
Porto -> Viseu	DLS (limite=4)	232	11	11	0.047
Porto -> Viseu	Greedy	133	2	2	0.088
Porto -> Viseu	A*	133	3	3	0.080

Os resultados evidenciam que UCS e A* obtiveram consistentemente os menores custos nos cenários avaliados, confirmando a expectativa teórica de otimalidade para A* com heurística adequada e UCS por construção. A Procura Sôfrega apresentou menor esforço de exploração em vários casos, mas com custo final superior, refletindo a ausência de consideração de g(n). A Profundidade Limitada mostrou dependência direta do limite escolhido, incluindo falha de solução quando o limite foi insuficiente

6.3 OCR

A componente OCR demonstrou utilidade prática na autenticação, com melhor desempenho em imagens de boa qualidade (boa iluminação, baixo ruído e enquadramento frontal). Em imagens com ruído, reflexos ou baixa resolução, observaram-se falhas de reconhecimento, mitigadas parcialmente por validação e normalização de formato.

6.4 LLM

A integração LLM forneceu descrições relevantes para a maioria das cidades testadas, acrescentando valor de apresentação e contexto ao percurso calculado. A qualidade textual depende do modelo local instalado e da disponibilidade do serviço.

6.5 Exportação JSON e CSV

A funcionalidade de exportação em JSON e CSV acrescenta rigor técnico e reprodutibilidade ao projeto, ao permitir guardar os resultados da comparação dos algoritmos num formato estruturado e reutilizável. A exportação em JSON preserva a estrutura completa dos dados (incluindo caminho, custo, iterações e métricas), sendo adequada para integração com outras aplicações, auditoria técnica e validação posterior. Já o formato CSV facilita análise tabular em ferramentas como Excel, Google Sheets ou software estatístico, apoiando a criação de gráficos e comparação quantitativa entre métodos. Em contexto académico, esta funcionalidade reforça a qualidade metodológica do trabalho, pois permite documentar evidência experimental, repetir análises e sustentar conclusões com dados objetivos.

```
[
  {
    "algorithm": "ucs",
    "path": [
      "Aveiro",
      "Leiria",
      "Santarém",
      "Évora",
      "Beja"
    ],
    "cost": 380,
    "iterations": [
      {
        "node": "Aveiro",
        "g": 0,
        "path": [
          "Aveiro"
        ]
      },
      {
        "node": "Coimbra",
        "g": 68,
        "path": [
          "Aveiro",
          "Coimbra"
        ]
      },
      {
        "node": "Porto",
        "g": 68,
        "path": [
          "Aveiro",
          "Porto"
        ]
      },
      {
        "node": "Viseu",
        "g": 95,
        "path": [
          "Aveiro",
          "Viseu"
        ]
      },
      {
        "node": "Leiria",
        "g": 115,
        "path": [
          "Aveiro",
          "Leiria"
```


Figura 19 - Exportação JSON

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	algorithm	distance_l	iterations	execution	expanded	path_node	found	path						
2	ucs	380	17	0.374	17	5	TRUE	Aveiro -> Leiria -> SantarÃ©m -> Ã‰vora -> Beja						
3	dls	508	11	0.119	11	5	TRUE	Aveiro -> Coimbra -> Castelo Branco -> Ã‰vora -> Beja						
4	greedy	472	5	0.388	5	5	TRUE	Aveiro -> Leiria -> Lisboa -> Ã‰vora -> Beja						
5	astar	380	6	0.184	6	5	TRUE	Aveiro -> Leiria -> SantarÃ©m -> Ã‰vora -> Beja						
6														
7														

Figura 20 - Exportação CSV

7 Discussão

A análise confirma a adequação da escolha dos algoritmos para fins pedagógicos e comparativos.

O Custo Uniforme constitui referência robusta de otimalidade, mas com maior custo exploratório.

O A* mostrou melhor compromisso global quando a heurística é informativa.

A Procura Sôfrega revelou-se útil para resposta rápida, mas com risco de solução subótima.

A Profundidade Limitada mostrou utilidade em cenários controlados, exigindo calibração de limite para evitar falhas de completude.

Do ponto de vista de engenharia, a separação entre backend, frontend e módulos de IA facilitou evolução incremental e manutenção. A adição de OCR e LLM elevou o projeto de uma implementação exclusivamente algorítmica para um produto interativo com maior valor aplicado.

As principais limitações observadas foram:

- Sensibilidade do OCR à qualidade da imagem.
- Dependência da disponibilidade local do LLM.
- Escala limitada do domínio geográfico analisado.

8 Conclusões e Trabalho Futuro

O projeto cumpriu os objetivos definidos, implementando e comparando quatro métodos de procura em contexto realista de navegação entre cidades. A integração de OCR e LLM demonstrou viabilidade técnica e benefício funcional para experiência do utilizador.

A avaliação experimental confirmou o alinhamento entre comportamento observado e previsões teóricas dos algoritmos.

Como trabalho futuro, propõem-se:

- Expansão do grafo para maior cobertura geográfica.
- Pré-processamento avançado de imagem para aumentar robustez do OCR.
- Benchmark sistemático com maior número de cenários e repetições.
- Integração de visualização geográfica avançada em mapa.
- Automatização de testes end-to-end da aplicação web.

Referências Bibliográficas

Koch, G., & Buitinck, L. (2023). FastAPI. https://fastapi.tiangolo.com/

React Developers. (2024). React documentation. https://react.dev/

Vite Team. (2024). Vite documentation. https://vitejs.dev/

JaidedAI. (2024). EasyOCR. GitHub. https://github.com/JaidedAI/EasyOCR

Ollama. (2024). Ollama documentation. https://ollama.com/librar

Python Software Foundation. (2024). json — JSON encoder and decoder. In Python 3 documentation. https://docs.python.org/3/library/json.html

The Apache Software Foundation. (2024). Common format and MIME type for comma-separated values (CSV). https://www.rfc-editor.org/rfc/rfc4180